

John Lee

Cloud Solutions Architect & DevOps Engineer in Seoul

Email iam@juany.dev

LinkedIn [in/juanylee](https://www.linkedin.com/in/juanylee)

GitHub [@ninejuan](https://github.com/ninejuan)

Tel +82 10 9048 9898

About

I'm a Solutions Architect who designs and operates cloud-native infrastructure on AWS and Kubernetes. At Algorix, I designed a VPC Lattice and Cilium-based architecture that reduced compute usage by 76.8 vCPU and 150 GiB per 300 Pods, saving roughly \$42K a year, and presented the work at AWS Summit Seoul 2026. I'm strongest at breaking complex systems down structurally and rebuilding them into architectures that are both operable and cost-efficient.

Work Experience

Sigmoid | Seoul, South Korea

Nov 2025 — Now

Founder

Founded Sigmoid to make cloud infrastructure easier to operate. I carry the product end to end, from planning and design through the front end, the back end, and the execution infrastructure that runs customer deployments. I built and piloted a tool that compiles an architecture drawn on a canvas into Terraform and deploys it to AWS, and I'm reshaping the product around what that pilot taught me about how teams actually work with their own infrastructure.

Algorix Corporation | Seoul, South Korea

Nov 2025 — Now

Solutions Architect

Designed a multi-cluster EKS networking architecture with Amazon VPC Lattice and Cilium ENI mode, replacing a sidecar-based mesh's operational overhead and resource cost. VPC Lattice handles cross-cluster connectivity, while Cilium's eBPF data plane provides network policy and traffic observability. Saved 76.8 vCPU and 150 GiB per 300 Pods; presented the work at AWS Summit Seoul 2026 (DEV305).

Projects

Improving AWS EKS multi-cluster networking: from VPC Peering to VPC Lattice and Cilium

Nov 2025 — Mar 2026

Amazon EKS, VPC Lattice, Cilium, Istio

As per-customer infrastructure and data isolation requirements grew, the EKS cluster setup hit the limits of VPC Peering and NLB. After adopting Istio, Sidecar resource usage and operational load increased, so I moved to a VPC Lattice and Cilium based Service Networking architecture, resolving connectivity and bootstrap issues. The result saved 76.8 vCPU and 150 GiB of memory per 300 Pods, cutting roughly \$42,000 per year.

Sigmoid, a Visual Infrastructure Platform

Nov 2025 — Sep 2026

Terraform, AWS Batch, NestJS, Next.js, React Flow

Across many startup meetings I found a shared pain point: AI made writing code easy, but deploying and operating it was still hard. I designed a Visual Infrastructure Platform where you draw infrastructure on a canvas and it auto-compiles and deploys to Terraform with stable IDs and a reference structure, and it secured pre-registrations from companies including Coupang and Deloitte. Running a pilot and sitting through the interviews that followed corrected the premise: hiding the infrastructure isn't enough, because a team still has to understand what it is running. I'm rebuilding the product around helping them get there rather than around keeping it out of sight.

ChaosPlane: building trustworthy fault experiments for Kubernetes

Mar 2026 — Jul 2026

Go, Kubernetes, Helm, gRPC

Built ChaosPlane, an open-source chaos engineering platform for Kubernetes, structured around an Operator, Executor, and Daemon so I could repeatedly verify how the cluster behaves under failure. It's organized into 6 repositories, including a CLI and Helm charts, and supports 20 fault experiments across Pod, Network, Node, and Stress categories. Pod-level faults trace down to their host-side veth and cgroup v2 path, so network and resource faults land on the Pod I target, not the whole node. Blast radius is capped with BlastRadiusPolicy, and an Abort Condition immediately halts a run the moment a steady-state probe drifts out of bounds. Recovery doesn't guess at the reverse command. Instead, it snapshots the pre-fault state and restores it as-is, so the outcome of a rollback is predictable.

Designing an EKS AI-driven Threat Detection and Response architecture

May 2026

Amazon EKS, Terraform, AWS Lambda, Claude, Falco, Tetragon

In EKS, I normalized heterogeneous detection events from GuardDuty, Falco, and Tetragon into a common schema, deduplicated them, and ran them through a four-stage Agent Chain: Summary → Triage → Solution → Remediation. Events are classified into a P1-P4 priority; Claude Haiku handles Summary and Triage, while Claude Sonnet — which generates the response and decides whether to execute it — handles Solution and Remediation, tuning cost and response speed by stage. P3/P4 events get remediated automatically, while P1/P2 only execute after Slack approval, with every analysis result, approval, and execution logged to DynamoDB as an audit trail. Kubernetes changes are never run by the agent directly — they go through an EKS MCP server with limited permissions, and a Destructive Action Guard in the execution layer blocks destructive actions until Pod snapshotting and Tetragon SIGKILL-labeling are complete.

Awards

WorldSkills Korea 61st National Competition — Cloud Computing (Bronze Medal, Joint 3rd Place)

Aug 2026

WorldSkills Korea / Human Resources Development Service of Korea

2026 WorldSkills Korea Local Competition - Cloud Computing (Gold Medal)

Apr 2026

WorldSkills Korea / Human Resources Development Service of Korea

Seoul Student AI Project Showcase — Grand Prize (Seoul Superintendent of Education Award)

Dec 2025

Seoul Metropolitan Office of Education

WorldSkills Korea 60th National Competition — Cloud Computing (Encouragement Award)

Sep 2025

WorldSkills Korea / Human Resources Development Service of Korea

2025 WorldSkills Korea Local Competition - Cloud Computing (Gold Medal)

Apr 2025

WorldSkills Korea / Human Resources Development Service of Korea

Speaking

Sidecar-less Networking Strategy for Multi-Cluster | Seoul, COEX

May 2026

AWS Summit Seoul 2026 — Community Track (DEV305)

Certifications

AWS Certified Solutions Architect – Associate

Dec 2025

Amazon Web Services (AWS) | Expires Dec 2028

Craftsman Information Processing

May 2025

Human Resources Development Service of Korea